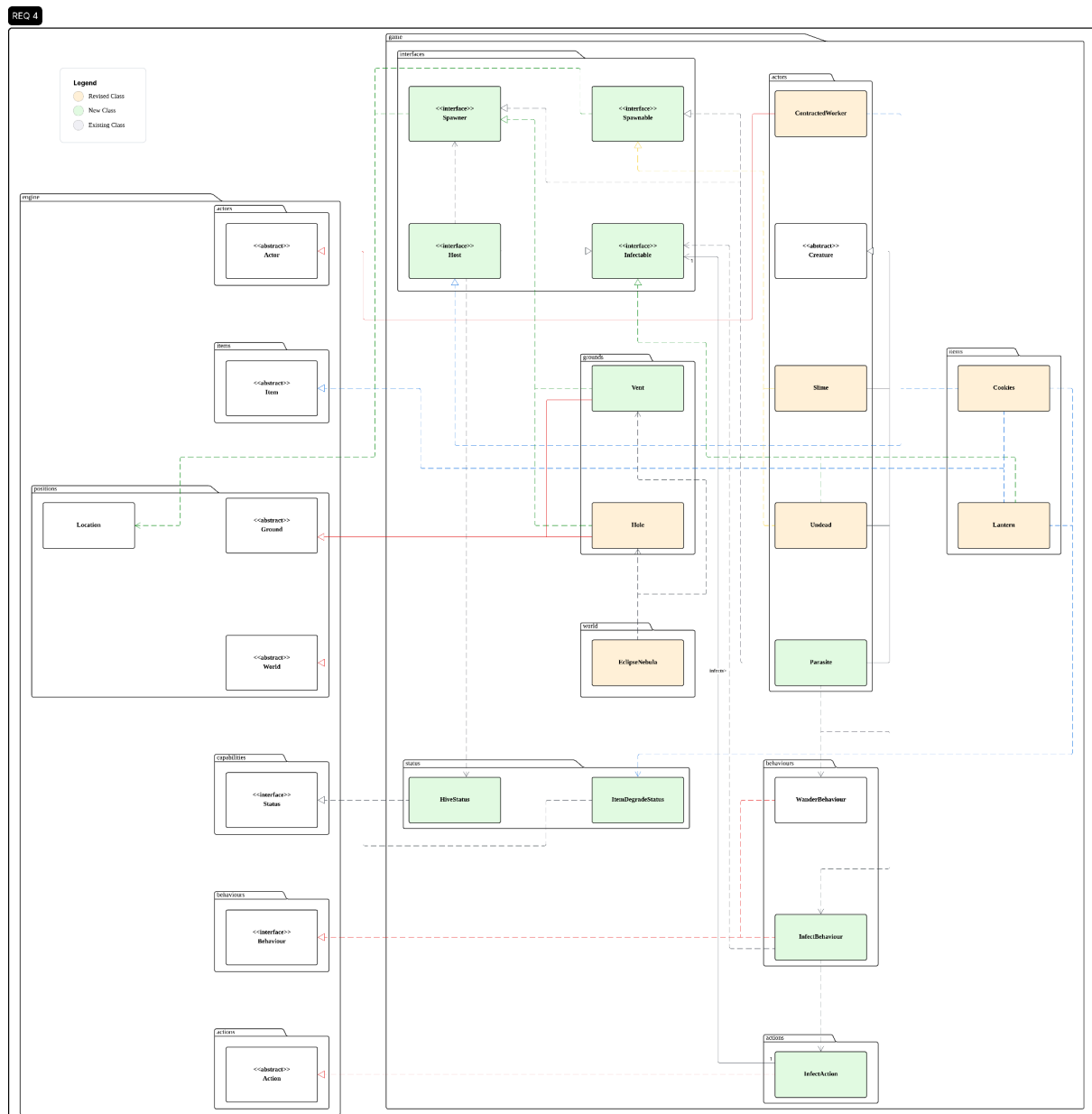


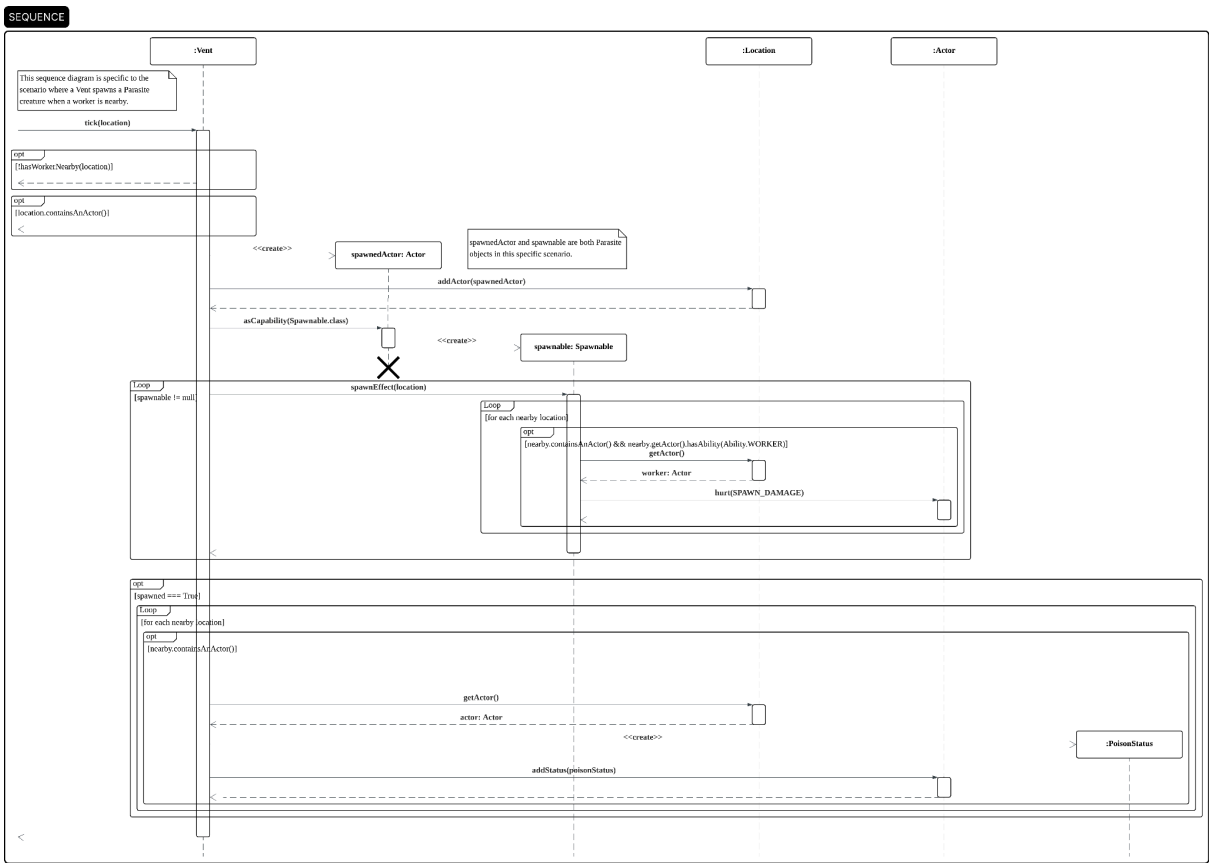
FIT2099 Assignment 2 Design Documentation

Design Diagrams

REQ 4 UML Class Diagram



REQ 4 UML Sequence Diagram



Design Rationale

Changes made to the design proposed for Assignment 1

Item Durability Tracking

Assignment 1 Design: Items tracked their usage through an instance counter field itemUses. Each item had a dedicated integer field that decremented as the item was used.

Pros	Cons
Simple to implement and understand	Inconsistent implementation across different items
Direct access to durability state without abstraction layers	Does not follow DRY principle—each item reimplements similar durability logic
	Limited extensibility for complex item behaviors

New Design: Items now use the ItemStatistics.DURABILITY enumeration combined with the engine's statistics API.

Pros	Cons
Follows DRY principle—all items use the same API	Added dependency on the engine's statistics system
Enhanced extensibility for status effects that interact with item statistics	
Type-safe enumeration prevents typos and invalid statistics	
Inheritance and polymorphism support for modifications to item durability	

The new design strongly aligns with SOLID principles, particularly DRY and the Open/Closed Principle. By centralizing durability tracking through `ItemStatistics.DURABILITY`, the system becomes more maintainable and extensible. This design enables future features such as damage resistance, item restoration, or other durability-based mechanics without modifying individual item classes. The statistics system is extensible (the `ItemStatistics` enum can be expanded), and the use of `StatisticOperations` (`INCREASE`, `DECREASE`, `UPDATE`) makes intent clear across the codebase.

All Items Made Portable

Assignment 1 Design: In previous implementations, not all items were portable.

Pros	Cons
	Inconsistent implementation across item types (some items could be carried, while others could not be dropped or picked up from the ground)
	Impossible to implement the slime spawn mechanic where hitting a worker drops all carried items

New Design: All items are now explicitly marked as portable via the `makePortable()` method, which is called in every item's constructor.

Pros	Cons
All items can be dropped and picked up from the ground as intended	
Enables clean implementation of slime spawn effect that drops items	
Reduces code duplication in item implementations in modifying counters	

Using the new design, this makes all items portable which aligns with the previous assignment requirements, as well as the Slime spawning mechanic. The Slime's spawn effect (forcing workers to

drop items) requires each item to be dropped from the actor via DropAction. Rather than implementing deletion from inventory and spawning on the ground for each item.

Changes made in Requirement 4

A. Infection Pipeline

Design 1: Infection as Capability-Based Interface

Infection is implemented as a capability interface Infectable. Any actor or item that implements Infectable can be infected. The parasite depends on InfectBehaviour, which discovers adjacent Infectable as targets and returns an InfectAction if found. InfectAction then triggers the infect() in Infectable where the infect() encapsulates the side effects from infection.

Pros	Cons
InfectBehaviour does not need to know specifics of how each target responds, which is a decoupled design.	Harder to debug than a centralised infection system
Easy to extend where new infectable entities simply implement the interface	
Able to leverage polymorphism where each entity can respond to infection differently	

Design 2: Infection as Centralised Action Handler

Centralise infection logic into a single InfectionHandler class that knows how to process infections for all entity types. The InfectBehaviour would return an InfectAction without discovering infectable types, and the action would delegate to the centralised handler to determine which target to infect and produce the side effect.

Pros	Cons
Simpler to understand the full infection flow in one handler class	Infection logic is centralised, making it hard to specify entity-specific rules
Easier to debug infection mechanics	Adding a new infectable entity requires modifying the handler
	Large switch/if-else chain as more entities are introduced
	Tight coupling between infection logic and all possible target types

B. Host & Hive Pipeline

Design 1: Host extends Infectable, HiveStatus to handle Host side effects

The Host interface extends Infectable and adds the `hive()` and `hiveEffect()`. When an entity becomes infected, it can transition to “host” status by calling `hive(spawner)`, which applies a `HiveStatus` status effect. The `HiveStatus` calls `hiveEffect()` on each game tick and produces any side effects of being a “host”.

Pros	Cons
Clear separation of responsibility as Host defines the contract, HiveStatus manages spawning logic	Host logic confined to game entities that are able to process statuses.
Decouples host behaviour from “host” entity implementations	

Design 2: “Host” entities to gain Ability.HOST when infected

The “host” game entities to enable the ability `HOST`. While an entity has the ability enabled, an internal method within each entity (`hive()`) will be called to produce side effects from the `tick()` method.

Pros	Cons
The use of <code>Ability.HOST</code> is a simple, discoverable flag	Duplicates spawning lifecycle/state across every host implementation
	Tightly couples host behaviour to concrete entity implementations
	No Host interface to define host contract which makes the code less readable

C. Creature Spawning Pipeline

Design 1: Spawner to handle Creation and Side Effects

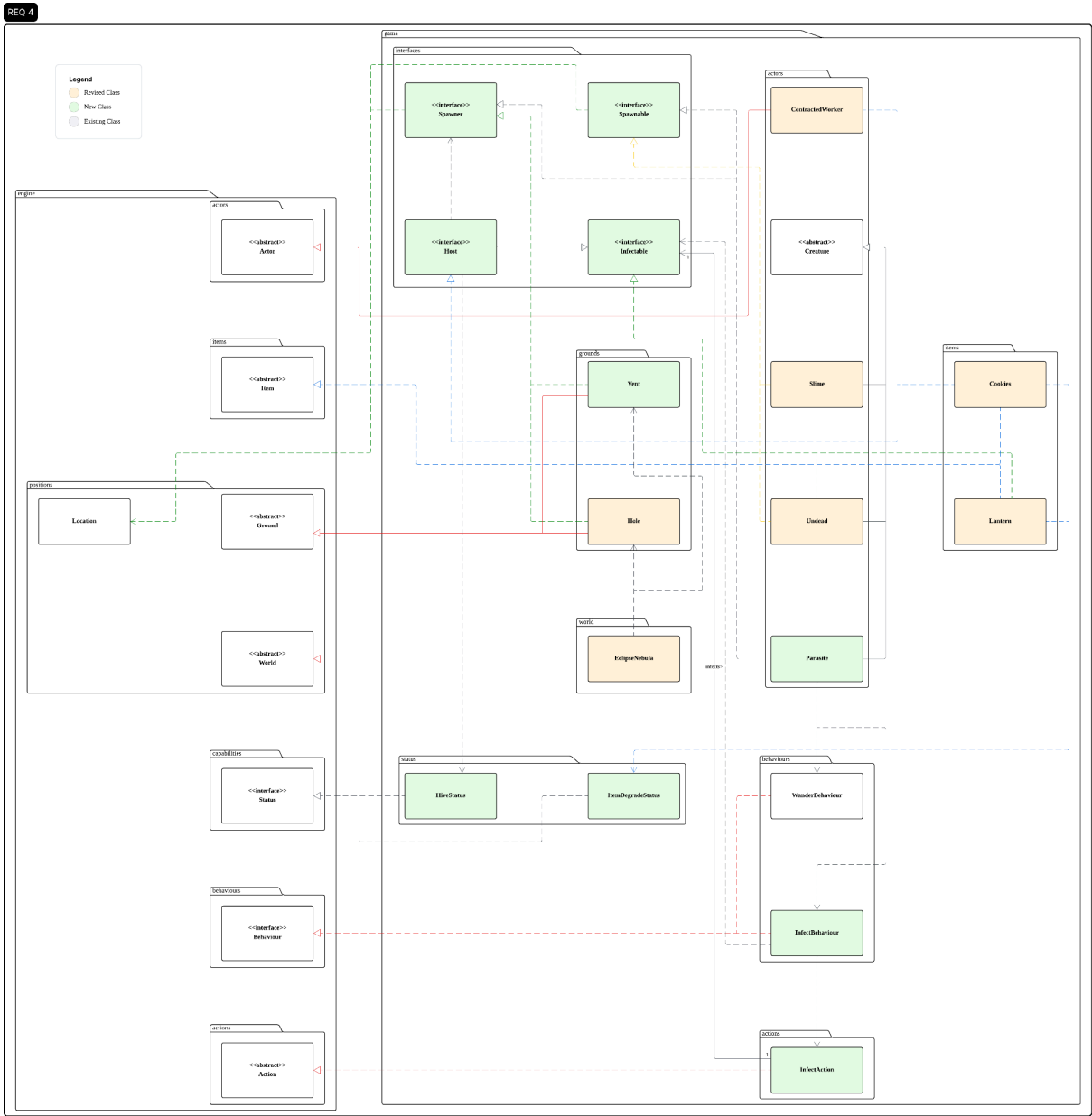
Spawner defines the contract for creating creature instances and executing side effects on spawned locations.

Pros	Cons
Easier to implement side effects as creatures are spawned	Couples creation with side effects, hard to reuse side effects with different spawners
	Duplicate side effect logic needed to be implemented for each spawner
	Harder to extend as each new creature side effect needed to be implemented for a new spawner

Design 2: Spawner as Factory with Spawnable as Lifecycle Hook

Creature spawning is divided into two interfaces (Spawner, Spawnable). Spawner defines the contract for creating creature instances. Spawnable defines the contract for executing side effects when a creature is spawned.

Pros	Cons
Same creature side effects can be reused in multiple systems where creatures were spawned	Side effects are implicit where spawners must remember to call spawnEffect() after adding the actor
Easier to test side effect behaviour without needing to test the entire pipeline	



The diagram above shows the final design of Requirement 4, which utilises Design 1 in Part A, Design 1 in Part B, and Design 2 in Part C.

The infection pipeline was implemented using the Infectable interface together with InfectBehaviour and InfectAction. This design separates the responsibility of detecting infection targets, triggering infection, and handling infection side effects into different components. InfectBehaviour is only responsible for identifying nearby infectable targets, while the concrete implementation of infect() determines how each entity reacts to infection. This follows the Single Responsibility Principle, as each class or interface manages only one concern. It also promotes the Open/Closed Principle because new infectable entities can be introduced by implementing the Infectable interface without modifying existing infection logic. Using polymorphism through the Infectable interface also avoids large instanceof or switch-based condition chains, improving maintainability and extensibility of the system.

The Host and Hive pipeline was implemented using the Host interface together with the HiveStatus status effect. The Host interface defines the contract for entities that can become hosts, while HiveStatus manages the lifecycle and recurring effects of the host state during each game tick. This design decouples the host spawning behaviour from the concrete entity implementation, reducing duplicated logic across multiple host entities. Separating the status effect from the entity itself improves cohesion and aligns with the Single Responsibility Principle because the entity is responsible for representing game behaviour, while HiveStatus is responsible for managing timed host effects. This implementation also improves extensibility as future host entities only need to implement the Host interface and define their own hiveEffect() behaviour without modifying existing classes, satisfying the Open/Closed Principle.

The creature spawning pipeline was implemented using separate Spawner and Spawnable interfaces. Spawner acts as a factory responsible for creating creature instances, while Spawnable defines lifecycle side effects that occur after spawning. Separating creature creation from spawn side effects reduces duplicated code and allows the same spawn effects to be reused across multiple spawning systems, improving adherence to the DRY principle. This design also improves modularity because spawning behaviour and spawn side effects can evolve independently. By depending on abstractions instead of concrete creature implementations, the spawning system follows the Dependency Inversion Principle, reducing coupling between spawning systems and spawned entities. Additionally, separating spawning responsibilities into dedicated interfaces improves maintainability and testing, as spawn side effects can be tested independently without requiring the entire spawning pipeline to execute.

The final design also demonstrates effective use of interfaces and abstractions throughout the system. Interfaces such as Infectable, Host, Spawner, and Spawnable provide clear behavioural contracts while allowing different concrete implementations to respond differently through polymorphism. This improves readability and extensibility of the system while avoiding duplicated implementations and tightly coupled logic. Overall, the design emphasises modularity, separation of responsibilities, extensibility, and maintainability through alignment with SOLID principles and DRY practices.